

Transformers

March 23, 2026

```
[2]: # Encoder
      # Embedding

      import torch
      import torch.nn as nn

      vocab_size = 1000
      d_model = 4

      embedding = nn.Embedding(vocab_size, d_model)

      tokens = torch.tensor([5, 27, 102])
      x = embedding(tokens)

      print(x)
      # tensor([[0.123, 0.456, 0.789, 0.012],
      #         [0.234, 0.567, 0.890, 0.123],
      #         [0.345, 0.678, 0.901, 0.234]])

      tensor([[ -1.8121, -0.4282,  0.4185, -0.7148],
              [ 0.3870, -0.1294, -0.9644,  0.3217],
              [ 0.8216,  1.0924, -1.0447,  1.0397]], grad_fn=<EmbeddingBackward0>)
```

```
[3]: # Positional encoding
      import math
      import torch

      # -----
      # 2 Positional Encoding
      # -----

      def get_positional_encoding(seq_len, d_model):
          PE = torch.zeros(seq_len, d_model)
          for pos in range(seq_len):
              for i in range(0, d_model, 2):
                  PE[pos, i] = math.sin(pos / (10000 ** (i / d_model)))
                  PE[pos, i+1] = math.cos(pos / (10000 ** (i / d_model)))
          return PE
```

```

PE = get_positional_encoding(seq_len=tokens.size(0), d_model=d_model)
print("Positional Encoding:")
print(PE)

```

```

Positional Encoding:
tensor([[ 0.0000,  1.0000,  0.0000,  1.0000],
        [ 0.8415,  0.5403,  0.0100,  0.9999],
        [ 0.9093, -0.4161,  0.0200,  0.9998]])

```

```

[4]: # -----
# 3 Add Embedding + Positional Encoding
# -----
x_with_position = x + PE
print("Embedding + Positional Encoding:")
print(x_with_position)

```

```

Embedding + Positional Encoding:
tensor([[ -1.8121,  0.5718,  0.4185,  0.2852],
        [ 1.2285,  0.4109, -0.9544,  1.3217],
        [ 1.7309,  0.6763, -1.0247,  2.0395]], grad_fn=<AddBackward0>)

```

```

[6]: # Self-Attention on Embeddings
import torch.nn.functional as F

# Use x_with_position as X
X = x_with_position # shape: (seq_len, d_model)
d_model = X.size(1)
d_k = d_model # for simplicity

# Linear projections (identity for demo)
Q = X
K = X
V = X

# Compute attention scores
scores = torch.matmul(Q, K.transpose(0,1)) / math.sqrt(d_k)
print("Scores:\n", scores)

# Softmax
weights = F.softmax(scores, dim=-1)
print("Attention Weights:\n", weights)

# Weighted sum
output = torch.matmul(weights, V)
print("Contextualized Output:\n", output)

```

```

Scores:
tensor([[ 1.9335, -1.0068, -1.2984],
        [-1.0068,  2.1678,  3.0388],

```

```

        [-1.2984,  3.0388,  4.3314]], grad_fn=<DivBackward0>)
Attention Weights:
  tensor([[0.9155, 0.0484, 0.0361],
          [0.0122, 0.2914, 0.6964],
          [0.0028, 0.2148, 0.7824]], grad_fn=<SoftmaxBackward0>)
Contextualized Output:
  tensor([[ -1.5369,  0.5678,  0.2999,  0.3988],
          [ 1.5413,  0.5977, -0.9866,  1.8089],
          [ 1.6130,  0.6190, -1.0056,  1.8804]], grad_fn=<MmBackward0>)

```

```

[7]: # multi-head attention

import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by \
↪num_heads"
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        # Learnable linear projections
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

    def forward(self, X):
        # X: (batch_size, seq_len, d_model)
        batch_size, seq_len, d_model = X.shape

        # Linear projections
        Q = self.W_Q(X)
        K = self.W_K(X)
        V = self.W_V(X)

        # Split into heads
        Q = Q.view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,2)
        K = K.view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,2)
        V = V.view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,2)

        # Scaled dot-product attention
        scores = torch.matmul(Q, K.transpose(-2,-1)) / math.sqrt(self.d_k)

```

```

        weights = F.softmax(scores, dim=-1)
        attention = torch.matmul(weights, V)

        # Concatenate heads
        attention = attention.transpose(1,2).contiguous().view(batch_size,
↪seq_len, d_model)

        # Final linear layer
        output = self.W_0(attention)
        return output

```

```

[8]: # apply multihead attention
# Prepare input: add batch dimension
X = x_with_position.unsqueeze(0) # shape: (1, seq_len, d_model)

# Create MHSA layer
mhsa = MultiHeadSelfAttention(d_model=4, num_heads=2)

# Forward pass
out = mhsa(X)
print("Multi-Head Self-Attention Output:\n", out)

```

Multi-Head Self-Attention Output:

```

tensor([[[ 0.0969, -0.4085,  0.8420, -0.1542],
          [ 0.0564, -0.4751,  0.4410, -0.1062],
          [ 0.0715, -0.4995,  0.3809, -0.0862]]], grad_fn=<ViewBackward0>)

```

```

[9]: # Add and norm
import torch
import torch.nn as nn

# Suppose 'X' is input to MHSA
# 'out' is output from MHSA
layer_norm1 = nn.LayerNorm(d_model)

# Residual connection + normalization
residual_out = X + out.squeeze(0) # remove batch dim if needed
normed_out = layer_norm1(residual_out)
print("After MHSA Add & Norm:\n", normed_out)

```

After MHSA Add & Norm:

```

tensor([[[ -1.5675,  0.1903,  1.2170,  0.1601],
          [ 1.0230, -0.6932, -1.2645,  0.9347],
          [ 0.8942, -0.5888, -1.3374,  1.0320]]],
        grad_fn=<NativeLayerNormBackward0>)

```

```

[10]: # Position wise feedforward

```

```

class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff) # W1, b1
        self.fc2 = nn.Linear(d_ff, d_model) # W2, b2

    def forward(self, X):
        # X: (seq_len, d_model)
        return self.fc2(F.relu(self.fc1(X)))

# Example usage
d_model = 4
d_ff = 8
ffn = PositionwiseFeedForward(d_model=d_model, d_ff=d_ff)

# Suppose normed_out is output from MHSA + Add & Norm
ffn_out = ffn(normed_out)
print("Feed-Forward Output:\n", ffn_out)

```

Feed-Forward Output:

```

tensor([[[[ 0.7284,  0.4030, -1.0379, -0.1591],
          [-0.2824,  0.3966, -0.0142,  0.1280],
          [-0.2676,  0.3641, -0.0383,  0.1637]]], grad_fn=<ViewBackward0>)]

```

```

[11]: import torch
import torch.nn as nn

# Suppose normed_out is input to FFN (after MHSA + Add & Norm)
# ffn_out is output of FFN
layer_norm2 = nn.LayerNorm(d_model)

# Residual connection + normalization
final_out = layer_norm2(normed_out + ffn_out)
print("After FFN Add & Norm:\n", final_out)

```

After FFN Add & Norm:

```

tensor([[[[-1.5781,  1.1695,  0.3750,  0.0335],
          [ 0.7428, -0.3842, -1.4513,  1.0928],
          [ 0.5897, -0.2892, -1.4776,  1.1771]]], grad_fn=<NativeLayerNormBackward0>)]

```

```

[14]: # -----
# Stack 5 more encoder layers after first completed layer
# -----

import torch
import torch.nn as nn

# Reuse your previously defined classes:

```

```

# - MultiHeadSelfAttention
# - PositionwiseFeedForward
# Assume `final_out` is output of first encoder layer
# Shape of final_out: (1, seq_len, d_model) or (seq_len, d_model)

# Ensure batch dimension exists
X = final_out
if X.dim() == 2: # (seq_len, d_model) → add batch dim
    X = X.unsqueeze(0) # now shape: (1, seq_len, d_model)

# Define EncoderLayer class (reuse from previous cell)
class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.mhsa = MultiHeadSelfAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.ffn = PositionwiseFeedForward(d_model, d_ff)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, X):
        # MHSA + Add & Norm
        mhsa_out = self.mhsa(X)
        X = self.norm1(X + mhsa_out)
        # FFN + Add & Norm
        ffn_out = self.ffn(X)
        X = self.norm2(X + ffn_out)
        return X

# Stack 5 more layers
num_additional_layers = 5
encoder_layers = nn.ModuleList(
    [EncoderLayer(d_model=4, num_heads=2, d_ff=8) for _ in
     ↪range(num_additional_layers)]
)

# Forward pass through remaining layers
for i, layer in enumerate(encoder_layers, start=2): # layer numbering starts
    ↪at 2
    X = layer(X)
    print(f"Output after Encoder Layer {i}:\n", X)

```

Output after Encoder Layer 2:

```

tensor([[[[-1.6409,  1.0162,  0.1131,  0.5116],
          [ 0.0364, -0.2058, -1.3192,  1.4886],
          [-0.0802, -0.1356, -1.2978,  1.5136]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Output after Encoder Layer 3:

```

tensor([[[[-1.6642,  0.9861,  0.2202,  0.4579],
          [-0.0982, -0.2916, -1.1888,  1.5786],
          [-0.2101, -0.2217, -1.1650,  1.5967]]]],
        grad_fn=<NativeLayerNormBackward0>)
Output after Encoder Layer 4:
tensor([[[[-1.6237,  1.0869,  0.1302,  0.4067],
          [-0.0418, -0.0490, -1.3673,  1.4582],
          [-0.1576,  0.0279, -1.3433,  1.4731]]]],
        grad_fn=<NativeLayerNormBackward0>)
Output after Encoder Layer 5:
tensor([[[[-1.6918,  0.7446,  0.2142,  0.7330],
          [ 0.1536,  0.0274, -1.4975,  1.3165],
          [ 0.0642,  0.0801, -1.4826,  1.3384]]]],
        grad_fn=<NativeLayerNormBackward0>)
Output after Encoder Layer 6:
tensor([[[[-1.7318,  0.5574,  0.5699,  0.6045],
          [ 0.6153, -0.5128, -1.3461,  1.2435],
          [ 0.5232, -0.4550, -1.3602,  1.2919]]]],
        grad_fn=<NativeLayerNormBackward0>)

```

```

[16]: # class DecoderLayer(nn.Module):
#       def __init__(self, d_model, num_heads, d_ff):
#           super().__init__()
#           self.self_attn = MultiHeadSelfAttention(d_model, num_heads)
#           self.enc_dec_attn = MultiHeadSelfAttention(d_model, num_heads)
#           self.norm1 = nn.LayerNorm(d_model)
#           self.norm2 = nn.LayerNorm(d_model)
#           self.norm3 = nn.LayerNorm(d_model)
#           self.ffn = PositionwiseFeedForward(d_model, d_ff)
#
#       def forward(self, X, encoder_out):
#           # 1 Masked Self-Attention + Add & Norm
#           self_attn_out = self.self_attn(X) # masking to be added later
#           X = self.norm1(X + self_attn_out)
#
#           # 2 Encoder-Decoder Attention + Add & Norm
#           enc_dec_out = self.enc_dec_attn(X, K=encoder_out, V=encoder_out)
#           X = self.norm2(X + enc_dec_out)
#
#           # 3 Feed-Forward + Add & Norm
#           ffn_out = self.ffn(X)
#           X = self.norm3(X + ffn_out)
#
#       return X

```

```

[17]: # class TransformerDecoder(nn.Module):
#       def __init__(self, num_layers, d_model, num_heads, d_ff):

```

```

#         super().__init__()
#         self.layers = nn.ModuleList(
#             [DecoderLayer(d_model, num_heads, d_ff) for _ in
#             ↪range(num_layers)]
#         )

#     def forward(self, X, encoder_out):
#         for layer in self.layers:
#             X = layer(X, encoder_out)
#         return X

```

```

[19]: import torch
import torch.nn as nn
import torch.nn.functional as F
import math

# -----
# Assume encoder output is ready (from previous encoder)
# -----
batch_size = 1
seq_len_enc = 3
d_model = 4
vocab_size = 1000

# Example encoder output (from your 6-layer encoder)
encoder_out = torch.rand(batch_size, seq_len_enc, d_model) # shape: (1,
↪seq_len_enc, d_model)

# -----
# Target embedding (decoder input)
# -----
seq_len_dec = 3
target_tokens = torch.tensor([1, 2, 3])
embedding = nn.Embedding(vocab_size, d_model)
target_embeddings = embedding(target_tokens).unsqueeze(0) # shape: (1,
↪seq_len_dec, d_model)

# -----
# Multi-Head Self-Attention
# -----
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.W_Q = nn.Linear(d_model, d_model)

```



```

        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

    def forward(self, X, K=None, V=None):
        K = X if K is None else K
        V = X if V is None else V

        batch_size, seq_len, d_model = X.shape
        Q = self.W_Q(X)
        K = self.W_K(K)
        V = self.W_V(V)

        # Split into heads
        Q = Q.view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,2)
        K = K.view(batch_size, K.size(1), self.num_heads, self.d_k).
        ↪transpose(1,2)
        V = V.view(batch_size, V.size(1), self.num_heads, self.d_k).
        ↪transpose(1,2)

        # Scaled dot-product attention
        scores = torch.matmul(Q, K.transpose(-2,-1)) / math.sqrt(self.d_k)
        weights = F.softmax(scores, dim=-1)
        attention = torch.matmul(weights, V)

        # Concatenate heads
        attention = attention.transpose(1,2).contiguous().view(batch_size, ↪
        ↪seq_len, d_model)
        output = self.W_O(attention)
        return output

# -----
# Position-wise Feedforward
# -----
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, X):
        return self.fc2(F.relu(self.fc1(X)))

# -----
# Decoder Layer
# -----
class DecoderLayer(nn.Module):

```

```

def __init__(self, d_model, num_heads, d_ff):
    super().__init__()
    self.self_attn = MultiHeadSelfAttention(d_model, num_heads)
    self.enc_dec_attn = MultiHeadSelfAttention(d_model, num_heads)
    self.norm1 = nn.LayerNorm(d_model)
    self.norm2 = nn.LayerNorm(d_model)
    self.norm3 = nn.LayerNorm(d_model)
    self.ffn = PositionwiseFeedForward(d_model, d_ff)

def forward(self, X, encoder_out):
    # Masked Self-Attention + Add & Norm
    self_attn_out = self.self_attn(X)
    X = self.norm1(X + self_attn_out)

    # Encoder-Decoder Attention + Add & Norm
    enc_dec_out = self.enc_dec_attn(X, K=encoder_out, V=encoder_out)
    X = self.norm2(X + enc_dec_out)

    # Feed-Forward + Add & Norm
    ffn_out = self.ffn(X)
    X = self.norm3(X + ffn_out)

    return X

# -----
# Decoder Stack
# -----
class TransformerDecoder(nn.Module):
    def __init__(self, num_layers, d_model, num_heads, d_ff):
        super().__init__()
        self.layers = nn.ModuleList([DecoderLayer(d_model, num_heads, d_ff) for
↪ _ in range(num_layers)])

    def forward(self, X, encoder_out):
        layer_outputs = []
        for layer in self.layers:
            X = layer(X, encoder_out)
            layer_outputs.append(X)
        return layer_outputs

# -----
# Instantiate 6-layer decoder
# -----
num_layers = 6
num_heads = 2
d_ff = 8

```

```

decoder = TransformerDecoder(num_layers=num_layers, d_model=d_model,
    ↪ num_heads=num_heads, d_ff=d_ff)
decoder_outputs = decoder(target_embeddings, encoder_out)

# Print output embeddings of each decoder layer
for i, out in enumerate(decoder_outputs, 1):
    print(f"Decoder Layer {i} Output:\n", out)

# -----
# Output Projection to Vocabulary
# -----
output_proj = nn.Linear(d_model, vocab_size)

# Final decoder output
decoder_out_last = decoder_outputs[-1]

# Compute logits and probabilities
logits = output_proj(decoder_out_last)
probs = F.softmax(logits, dim=-1)

print("\nProbabilities shape:", probs.shape)
print("Example probabilities for first token:\n", probs[0,0])

```

Decoder Layer 1 Output:

```

tensor([[[[-0.7051,  0.9884,  0.9732, -1.2565],
          [-1.4204,  1.3868,  0.1880, -0.1544],
          [ 1.0783,  0.7287, -0.3232, -1.4838]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Decoder Layer 2 Output:

```

tensor([[[[-1.3543,  0.6905,  1.1889, -0.5251],
          [-1.7026,  0.8060,  0.6058,  0.2908],
          [ 0.0945,  1.0814,  0.4441, -1.6200]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Decoder Layer 3 Output:

```

tensor([[[[-0.9587, -0.5402,  1.6620, -0.1631],
          [-1.5516, -0.1868,  0.7167,  1.0217],
          [ 1.0639, -0.0460,  0.5746, -1.5924]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Decoder Layer 4 Output:

```

tensor([[[[-0.6554, -0.8753,  1.6689, -0.1381],
          [-1.3321, -0.5920,  0.8524,  1.0717],
          [ 1.2301, -0.2925,  0.5215, -1.4592]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Decoder Layer 5 Output:

```

tensor([[[[ 0.2162, -1.6098,  1.1389,  0.2547],
          [-0.6895, -1.1480,  0.4097,  1.4278],
          [ 1.6776, -0.4601, -0.2676, -0.9499]]],
        grad_fn=<NativeLayerNormBackward0>)]

```

Decoder Layer 6 Output:

```
tensor([[[ 0.7845, -1.5414,  0.9793, -0.2223],
          [-1.0231, -0.6002,  0.0130,  1.6103],
          [ 1.5261,  0.2512, -0.7695, -1.0078]]],
        grad_fn=<NativeLayerNormBackward0>)
```

Probabilities shape: torch.Size([1, 3, 1000])

Example probabilities for first token:

```
tensor([0.0019, 0.0007, 0.0013, 0.0006, 0.0023, 0.0028, 0.0022, 0.0011, 0.0007,
        0.0014, 0.0035, 0.0006, 0.0005, 0.0008, 0.0003, 0.0003, 0.0004, 0.0010,
        0.0020, 0.0025, 0.0011, 0.0011, 0.0008, 0.0010, 0.0006, 0.0003, 0.0005,
        0.0011, 0.0018, 0.0012, 0.0006, 0.0014, 0.0005, 0.0007, 0.0004, 0.0027,
        0.0006, 0.0007, 0.0009, 0.0005, 0.0005, 0.0008, 0.0009, 0.0003, 0.0004,
        0.0005, 0.0015, 0.0007, 0.0018, 0.0006, 0.0006, 0.0009, 0.0020, 0.0013,
        0.0002, 0.0026, 0.0017, 0.0009, 0.0016, 0.0012, 0.0014, 0.0007, 0.0015,
        0.0015, 0.0012, 0.0008, 0.0003, 0.0009, 0.0026, 0.0014, 0.0024, 0.0004,
        0.0006, 0.0010, 0.0009, 0.0006, 0.0020, 0.0004, 0.0008, 0.0004, 0.0020,
        0.0005, 0.0004, 0.0021, 0.0007, 0.0020, 0.0012, 0.0007, 0.0016, 0.0016,
        0.0003, 0.0013, 0.0006, 0.0009, 0.0004, 0.0005, 0.0003, 0.0006, 0.0022,
        0.0003, 0.0005, 0.0014, 0.0005, 0.0009, 0.0006, 0.0018, 0.0006, 0.0009,
        0.0009, 0.0014, 0.0014, 0.0006, 0.0004, 0.0029, 0.0022, 0.0004, 0.0004,
        0.0004, 0.0011, 0.0006, 0.0011, 0.0005, 0.0006, 0.0010, 0.0007, 0.0011,
        0.0003, 0.0009, 0.0007, 0.0004, 0.0010, 0.0006, 0.0005, 0.0004, 0.0012,
        0.0019, 0.0017, 0.0004, 0.0006, 0.0004, 0.0012, 0.0006, 0.0017, 0.0006,
        0.0005, 0.0003, 0.0013, 0.0009, 0.0008, 0.0008, 0.0007, 0.0012, 0.0020,
        0.0003, 0.0006, 0.0016, 0.0015, 0.0024, 0.0016, 0.0012, 0.0005, 0.0007,
        0.0007, 0.0011, 0.0008, 0.0009, 0.0017, 0.0003, 0.0007, 0.0012, 0.0006,
        0.0012, 0.0021, 0.0006, 0.0024, 0.0004, 0.0003, 0.0011, 0.0003, 0.0015,
        0.0018, 0.0004, 0.0006, 0.0013, 0.0008, 0.0006, 0.0010, 0.0003, 0.0013,
        0.0007, 0.0012, 0.0006, 0.0005, 0.0004, 0.0008, 0.0035, 0.0017, 0.0005,
        0.0017, 0.0007, 0.0026, 0.0013, 0.0007, 0.0006, 0.0025, 0.0009, 0.0005,
        0.0005, 0.0007, 0.0002, 0.0008, 0.0006, 0.0002, 0.0003, 0.0003, 0.0003,
        0.0006, 0.0017, 0.0007, 0.0004, 0.0025, 0.0011, 0.0018, 0.0009, 0.0011,
        0.0011, 0.0007, 0.0008, 0.0015, 0.0009, 0.0016, 0.0006, 0.0021, 0.0011,
        0.0003, 0.0002, 0.0006, 0.0025, 0.0003, 0.0015, 0.0022, 0.0007, 0.0012,
        0.0010, 0.0014, 0.0006, 0.0011, 0.0008, 0.0007, 0.0009, 0.0006, 0.0029,
        0.0011, 0.0009, 0.0007, 0.0009, 0.0006, 0.0034, 0.0010, 0.0011, 0.0005,
        0.0007, 0.0008, 0.0003, 0.0016, 0.0014, 0.0027, 0.0003, 0.0013, 0.0009,
        0.0013, 0.0007, 0.0007, 0.0006, 0.0008, 0.0003, 0.0005, 0.0007, 0.0008,
        0.0009, 0.0010, 0.0003, 0.0011, 0.0007, 0.0010, 0.0009, 0.0020, 0.0005,
        0.0006, 0.0006, 0.0003, 0.0012, 0.0030, 0.0004, 0.0021, 0.0010, 0.0005,
        0.0006, 0.0007, 0.0008, 0.0007, 0.0015, 0.0004, 0.0004, 0.0014, 0.0005,
        0.0007, 0.0005, 0.0010, 0.0003, 0.0002, 0.0009, 0.0002, 0.0008, 0.0024,
        0.0025, 0.0010, 0.0005, 0.0007, 0.0008, 0.0008, 0.0008, 0.0006, 0.0030,
        0.0005, 0.0005, 0.0006, 0.0032, 0.0002, 0.0006, 0.0006, 0.0003, 0.0017,
        0.0018, 0.0008, 0.0004, 0.0004, 0.0024, 0.0009, 0.0005, 0.0004, 0.0004,
        0.0040, 0.0006, 0.0021, 0.0002, 0.0002, 0.0020, 0.0014, 0.0007, 0.0013,
        0.0014, 0.0004, 0.0002, 0.0005, 0.0010, 0.0009, 0.0021, 0.0003, 0.0019,
```

0.0026, 0.0013, 0.0005, 0.0001, 0.0004, 0.0005, 0.0005, 0.0005, 0.0005,
0.0010, 0.0002, 0.0004, 0.0019, 0.0006, 0.0006, 0.0009, 0.0010, 0.0007,
0.0007, 0.0008, 0.0013, 0.0009, 0.0003, 0.0004, 0.0008, 0.0007, 0.0035,
0.0004, 0.0002, 0.0013, 0.0030, 0.0010, 0.0006, 0.0006, 0.0010, 0.0010,
0.0012, 0.0008, 0.0007, 0.0016, 0.0004, 0.0009, 0.0007, 0.0009, 0.0004,
0.0003, 0.0041, 0.0004, 0.0003, 0.0006, 0.0011, 0.0009, 0.0026, 0.0019,
0.0004, 0.0005, 0.0006, 0.0004, 0.0004, 0.0006, 0.0005, 0.0004, 0.0011,
0.0018, 0.0003, 0.0016, 0.0016, 0.0009, 0.0017, 0.0002, 0.0008, 0.0013,
0.0004, 0.0027, 0.0010, 0.0009, 0.0009, 0.0031, 0.0002, 0.0032, 0.0015,
0.0003, 0.0015, 0.0008, 0.0004, 0.0004, 0.0018, 0.0004, 0.0008, 0.0013,
0.0007, 0.0016, 0.0004, 0.0028, 0.0005, 0.0022, 0.0003, 0.0011, 0.0006,
0.0005, 0.0011, 0.0007, 0.0010, 0.0002, 0.0007, 0.0009, 0.0023, 0.0011,
0.0023, 0.0006, 0.0030, 0.0005, 0.0012, 0.0004, 0.0003, 0.0004, 0.0006,
0.0004, 0.0004, 0.0007, 0.0008, 0.0003, 0.0004, 0.0002, 0.0004, 0.0017,
0.0027, 0.0028, 0.0004, 0.0010, 0.0033, 0.0005, 0.0013, 0.0003, 0.0005,
0.0009, 0.0010, 0.0024, 0.0022, 0.0006, 0.0009, 0.0029, 0.0011, 0.0005,
0.0008, 0.0008, 0.0009, 0.0004, 0.0008, 0.0017, 0.0012, 0.0010, 0.0018,
0.0005, 0.0003, 0.0016, 0.0007, 0.0009, 0.0012, 0.0005, 0.0022, 0.0004,
0.0003, 0.0011, 0.0003, 0.0004, 0.0002, 0.0002, 0.0008, 0.0017, 0.0024,
0.0011, 0.0006, 0.0012, 0.0033, 0.0013, 0.0007, 0.0014, 0.0011, 0.0011,
0.0003, 0.0005, 0.0002, 0.0004, 0.0012, 0.0012, 0.0004, 0.0008, 0.0006,
0.0006, 0.0005, 0.0003, 0.0017, 0.0009, 0.0007, 0.0004, 0.0008, 0.0009,
0.0007, 0.0003, 0.0042, 0.0013, 0.0004, 0.0013, 0.0012, 0.0010, 0.0010,
0.0013, 0.0007, 0.0010, 0.0002, 0.0008, 0.0017, 0.0003, 0.0004, 0.0003,
0.0003, 0.0003, 0.0019, 0.0006, 0.0003, 0.0016, 0.0010, 0.0005, 0.0004,
0.0013, 0.0015, 0.0037, 0.0005, 0.0008, 0.0004, 0.0009, 0.0006, 0.0010,
0.0003, 0.0026, 0.0006, 0.0036, 0.0005, 0.0004, 0.0005, 0.0008, 0.0015,
0.0008, 0.0019, 0.0005, 0.0006, 0.0005, 0.0002, 0.0013, 0.0006, 0.0007,
0.0008, 0.0007, 0.0008, 0.0005, 0.0018, 0.0008, 0.0006, 0.0026, 0.0005,
0.0004, 0.0038, 0.0010, 0.0010, 0.0008, 0.0021, 0.0007, 0.0006, 0.0007,
0.0003, 0.0007, 0.0005, 0.0020, 0.0003, 0.0007, 0.0008, 0.0012, 0.0014,
0.0011, 0.0020, 0.0003, 0.0006, 0.0005, 0.0015, 0.0006, 0.0004, 0.0030,
0.0012, 0.0009, 0.0024, 0.0010, 0.0006, 0.0012, 0.0005, 0.0003, 0.0011,
0.0033, 0.0005, 0.0004, 0.0010, 0.0020, 0.0005, 0.0009, 0.0006, 0.0007,
0.0008, 0.0012, 0.0011, 0.0026, 0.0021, 0.0008, 0.0003, 0.0019, 0.0011,
0.0025, 0.0011, 0.0004, 0.0007, 0.0005, 0.0019, 0.0003, 0.0016, 0.0022,
0.0014, 0.0003, 0.0006, 0.0005, 0.0008, 0.0007, 0.0013, 0.0004, 0.0032,
0.0007, 0.0011, 0.0010, 0.0008, 0.0013, 0.0012, 0.0004, 0.0011, 0.0009,
0.0011, 0.0006, 0.0008, 0.0006, 0.0006, 0.0013, 0.0003, 0.0005, 0.0014,
0.0002, 0.0023, 0.0008, 0.0008, 0.0007, 0.0039, 0.0004, 0.0006, 0.0004,
0.0013, 0.0005, 0.0006, 0.0004, 0.0021, 0.0010, 0.0005, 0.0008, 0.0004,
0.0013, 0.0009, 0.0009, 0.0004, 0.0021, 0.0009, 0.0010, 0.0003, 0.0003,
0.0020, 0.0005, 0.0005, 0.0026, 0.0002, 0.0012, 0.0004, 0.0011, 0.0007,
0.0015, 0.0003, 0.0005, 0.0005, 0.0005, 0.0008, 0.0013, 0.0007, 0.0006,
0.0005, 0.0012, 0.0002, 0.0004, 0.0008, 0.0008, 0.0005, 0.0002, 0.0013,
0.0006, 0.0019, 0.0005, 0.0017, 0.0014, 0.0018, 0.0050, 0.0013, 0.0004,
0.0011, 0.0005, 0.0007, 0.0009, 0.0008, 0.0011, 0.0008, 0.0013, 0.0012,
0.0006, 0.0005, 0.0003, 0.0012, 0.0004, 0.0006, 0.0005, 0.0005, 0.0021,

```

0.0010, 0.0014, 0.0004, 0.0010, 0.0003, 0.0002, 0.0009, 0.0007, 0.0005,
0.0005, 0.0006, 0.0015, 0.0010, 0.0004, 0.0002, 0.0014, 0.0002, 0.0006,
0.0028, 0.0009, 0.0015, 0.0017, 0.0002, 0.0002, 0.0006, 0.0002, 0.0006,
0.0013, 0.0005, 0.0012, 0.0006, 0.0007, 0.0011, 0.0004, 0.0004, 0.0012,
0.0013, 0.0012, 0.0008, 0.0024, 0.0009, 0.0005, 0.0011, 0.0008, 0.0003,
0.0010, 0.0006, 0.0004, 0.0007, 0.0015, 0.0004, 0.0017, 0.0012, 0.0010,
0.0009, 0.0005, 0.0004, 0.0015, 0.0007, 0.0006, 0.0003, 0.0011, 0.0016,
0.0009, 0.0019, 0.0019, 0.0005, 0.0003, 0.0015, 0.0006, 0.0019, 0.0008,
0.0004, 0.0007, 0.0020, 0.0008, 0.0012, 0.0011, 0.0003, 0.0006, 0.0006,
0.0011, 0.0011, 0.0003, 0.0006, 0.0007, 0.0027, 0.0010, 0.0002, 0.0012,
0.0007, 0.0019, 0.0009, 0.0008, 0.0004, 0.0010, 0.0010, 0.0023, 0.0012,
0.0003, 0.0004, 0.0008, 0.0011, 0.0017, 0.0004, 0.0012, 0.0003, 0.0006,
0.0004, 0.0004, 0.0003, 0.0005, 0.0010, 0.0017, 0.0016, 0.0008, 0.0018,
0.0006, 0.0004, 0.0010, 0.0007, 0.0016, 0.0004, 0.0003, 0.0006, 0.0006,
0.0030, 0.0017, 0.0004, 0.0007, 0.0008, 0.0008, 0.0004, 0.0003, 0.0006,
0.0006, 0.0010, 0.0014, 0.0015, 0.0017, 0.0011, 0.0018, 0.0007, 0.0018,
0.0004, 0.0004, 0.0002, 0.0015, 0.0005, 0.0002, 0.0030, 0.0005, 0.0004,
0.0010, 0.0029, 0.0008, 0.0007, 0.0002, 0.0003, 0.0029, 0.0010, 0.0003,
0.0009, 0.0007, 0.0022, 0.0019, 0.0038, 0.0002, 0.0003, 0.0006, 0.0015,
0.0014, 0.0006, 0.0004, 0.0006, 0.0006, 0.0009, 0.0011, 0.0011, 0.0013,
0.0013, 0.0005, 0.0006, 0.0023, 0.0003, 0.0024, 0.0006, 0.0011, 0.0006,
0.0014, 0.0008, 0.0003, 0.0007, 0.0003, 0.0014, 0.0009, 0.0017, 0.0017,
0.0012, 0.0016, 0.0004, 0.0017, 0.0011, 0.0005, 0.0004, 0.0015, 0.0027,
0.0008], grad_fn=<SelectBackward0>)

```

```

[20]: # -----
# Autoregressive Decoding (Greedy)
# -----
max_len = 5 # maximum target sequence length
start_token = 0
end_token = 999 # example EOS token

# Initialize target sequence with start token
generated = [start_token]

for t in range(max_len):
    # Convert current sequence to tensor and embed
    tgt_seq = torch.tensor(generated).unsqueeze(0) # shape: (1, current_len)
    tgt_emb = embedding(tgt_seq) # shape: (1, current_len, d_model)

    # Run decoder
    decoder_outputs = decoder(tgt_emb, encoder_out)
    last_output = decoder_outputs[-1] # last layer output

    # Project to vocabulary logits
    logits = output_proj(last_output)

```

```
# Take the last token's probabilities
probs = F.softmax(logits, dim=-1)
next_token = torch.argmax(probs[0, -1]).item()

generated.append(next_token)

if next_token == end_token:
    break

print("Generated token IDs:", generated)
```

Generated token IDs: [0, 439, 647, 970, 970, 970]

[]: